



Identified Topics

No YouTube video summary or transcript has been provided yet. Please share the summary/transcript containing the DSA topics, and I'll immediately:

1. **Extract ALL main topics** clearly in a list
2. **Start teaching the FIRST topic** in complete depth using the exact structure requested
3. **Move sequentially** only after fully completing each topic

Once you provide the content, we'll dive deep into FAANG-level preparation – focusing on intuition, code, dry runs, and interview tricks that land high packages at top product companies.

Ready when you are! Just paste the summary/transcript. 🚀

STEP 1: Topic Identification

From the YouTube video summary, here are ALL the main topics extracted clearly:

1. **Foundations of Complexity** (Time Complexity, Space Complexity, Worst-Case Scenario, Growth Rates)
2. **Asymptotic Notations** (Big-O, Big-Omega, Big-Theta, Little-o, Little-omega)
3. **Complexity in Practical Programming** (Space Complexity details, Recursive Programs & Call Stack)
4. **Solving Recurrence Relations** (Divide-and-Conquer with Akra-Bazzi, Homogeneous Linear Relations, Non-Homogeneous Linear Relations)
5. **Advanced Concepts** (NP-Completeness)

Now starting with **Topic 1** in complete depth...

#####

TOPIC: Foundations of Complexity

#####

1. Simple & Detailed Theory (Teacher Style)

Imagine you're planning a road trip from Kolhapur to Mumbai. **Time Complexity** is like asking "How long will this trip take in the worst traffic?" – it's about execution speed (how many operations your code performs). **Space Complexity** asks "How much luggage can my car hold?" – it's about memory usage.

Why do we care? Different computers run at different speeds, but complexity tells us how our algorithm scales with bigger inputs (like $n=10$ vs $n=1$ million).

Step-by-step intuition:

- **Step 1:** Count "basic operations" (like adding numbers, comparisons) – each is like 1 second of work.
- **Step 2:** See how this count grows as input size n increases.
- **Step 3:** Focus on **worst-case** because in interviews, they ask "What's the MAXIMUM time this could take?" – guarantees reliability.

WHY used: Without it, you might write code that works for small data but crashes/freezes on large real-world data (like Google's search index).

2. Core Concepts (Deep Understanding)

Time Complexity: Number of basic operations as function of input size n . Measures speed indirectly via operations.

Types of Cases:

- **Worst Case (Big-O):** Upper bound, slowest input (e.g., linear search for absent element: n comparisons).^{[11][12]}
- **Best Case:** Fastest (e.g., first match: 1 comparison) – rarely used.
- **Average Case:** Expected over random inputs – hard to compute exactly.

Growth Rates (ignoring constants/low terms):

- $O(1)$: Constant – like array access.
- $O(\log n)$: Halving search space (binary search).
- $O(n)$: Linear scan.

- $O(n \log n)$: Sorting.
- $O(n^2)$: Nested loops.
- $O(2^n)$: Exponential – avoid!

Why ignore constants? For $n \rightarrow \infty$, dominant term matters: $3n^2 + 2n \approx O(n^2)$.

Space Complexity: Total memory = variables + data structures + recursion stack. Auxiliary space excludes input.

Key Detail: Input always counts in space analysis unless specified as "auxiliary only."

3. Examples (Very Important)

Example 1: Simple Loop

```
int sum = 0;
for(int i=0; i<n; i++) sum += arr[i];
```

- Time: n additions $\rightarrow O(n)$
- Space: $O(1)$ (only sum, i variables)

Example 2: Nested Loops

```
for(int i=0; i<n; i++)
  for(int j=0; j<n; j++)
    // do something  $O(1)$ 
```

- Time: $n * n = O(n^2)$
- Space: $O(1)$

Example 3: Binary Search

```
int low=0, high=n-1;
while(low <= high){
  int mid = (low+high)/2;
  if(arr[mid] == target) return mid;
  else if(arr[mid] < target) low = mid+1;
  else high = mid-1;
```

```
}
```

- Time: $\log_2 n$ comparisons (halves each step) $\rightarrow O(\log n)$
- Space: $O(1)$

4. Important Interview Notes ⚠

- **Most Asked:** "What's the time/space complexity?" – Always say worst-case first: " $O(n)$ time, $O(1)$ space."
- **Tricky:** Space includes recursion stack! Iterative > recursive for space.
- **Confusion:** Don't say " $3n$ is $O(3n)$ " – always drop constants: $O(n)$.
- **FAANG Tip:** Compare growth: " $O(n \log n)$ better than $O(n^2)$ for $n=10^6$ " (10M vs 1T operations).
- **Question:** "Why worst-case over average?" – Ans: Guarantees performance; average assumes input distribution.

5. Problem Identification

No specific problems in summary for this topic. Adding **standard FAANG-level problems** for complexity analysis:

1. **Two Sum** (Array, target sum – analyze brute vs hashmap)
2. **Fibonacci** (Recursive vs iterative complexity)
3. **Find Duplicate** (Array with duplicates)

6. Solve Each Problem (VERY IMPORTANT)

◇ Problem 1: Two Sum

Problem Statement: Given array `nums` & target, return indices `i, j` where `nums[i] + nums[j] = target`. Assume exactly one solution. Constraints: $n \leq 10^4$.

Intuition (How to Think): Brute: check all pairs. Optimized: Use hashmap for $O(1)$ lookups.

Pattern Used: Hashing / Trade space for time.

Brute Force Approach

- Check all i, j pairs: $O(n^2)$ time, $O(1)$ space.

```
public int[] twoSum(int[] nums, int target) {
    for(int i=0; i<nums.length; i++)
        for(int j=i+1; j<nums.length; j++)
            if(nums[i]+nums[j]==target)
                return new int[]{i,j};
    return new int[]{};
}
```

Why inefficient: $n=10^4 \rightarrow 50M$ operations $\rightarrow$ TLE.

Optimized Approach

- Key idea: Store seen numbers & target - seen.
- Time: $O(n)$, Space: $O(n)$

Code (Java Preferred)

```
public int[] twoSum(int[] nums, int target) {
    Map<Integer, Integer> map = new HashMap<>(); // num -> index

    for(int i=0; i<nums.length; i++) {
        int complement = target - nums[i];
        if(map.containsKey(complement))
            return new int[]{map.get(complement), i};
        map.put(nums[i], i); // store after check
    }
    return new int[]{};
}
```

Detailed Dry Run ★

Input: nums=[1,3,12,14,15], target=9

$i=0$: complement=7, map={}, not found $\rightarrow$ map={2:0}

i=1: complement=2, map has 2 → return^[16]

WHY: Hashmap gives $O(1)$ lookup vs nested loops.

Edge Cases & Mistakes

- Duplicate numbers: Store after check prevents using same index.
- No solution: Assume exists per problem.
- Mistake: Forgetting `map.clear()` in leetcode follow-ups.

◇ Problem 2: Fibonacci

Problem Statement: Compute nth Fibonacci (0,1,1,2,3,5...). $n \leq 40$.

Intuition: Recursive tree branches exponentially.

Pattern Used: DP / Memoization.

Brute Force (Recursive)

```
int fib(int n) {
    if(n<=1) return n;
    return fib(n-1) + fib(n-2); //  $O(2^n)$  time,  $O(n)$  space (stack)
}
```

Why inefficient: `fib(40)` = 102M calls!

Optimized (Iterative)

```
public int fib(int n) {
    if(n<=1) return n;
    int prev2=0, prev1=1;
    for(int i=2; i<=n; i++) {
        int curr = prev1 + prev2;
        prev2 = prev1;
        prev1 = curr;
    }
    return prev1; //  $O(n)$  time,  $O(1)$  space
}
```

Detailed Dry Run ★

n=4:

i=2: curr=1+0=1, prev2=1, prev1=1

i=3: curr=1+1=2, prev2=1, prev1=2

i=4: curr=2+1=3, prev2=2, prev1=3 → return 3

Edge Cases: n=0→0, n=1→1. Mistake: int overflow for large n.

◇ Problem 3: Find Duplicate

Problem Statement: nums[1..n] has 1 duplicate, return it. $1 \leq \text{nums}[i] \leq n$, O(1) space.

Intuition: Brute sort $O(n \log n)$, but O(1) space → Floyd's Cycle Detection (treat as linked list).

Pattern Used: Two Pointers / Cycle Detection.

Code (Optimized)

```
int findDuplicate(int[] nums) {
    int slow = nums[0], fast = nums[0];
    do {
        slow = nums[slow];
        fast = nums[nums[fast]];
    } while(slow != fast);

    slow = nums[0];
    while(slow != fast) {
        slow = nums[slow];
        fast = nums[fast];
    }
    return slow;
}
```

Detailed Dry Run: nums=[14][17][18][16]

slow=1, fast=1

Loop1: slow=3, fast=2→2

Loop2: slow=4, fast=2→2 → meet at 2

Phase2: slow=1→3→4→2 meet fast.

7. Topic Summary (Quick Revision)

Key Points: Time=operations vs n, Space=memory. Worst-case guarantee. Drop constants.

Patterns: Count loops/recursion depth.

Tricks: Recursion space= $O(\text{depth})$. Hashing trades space for time. Always specify both complexities.

Next: Moving to Topic 2 after your questions/confirmation on this! 🚀

**

next

#####

TOPIC: Asymptotic Notations

#####

1. Simple & Detailed Theory (Teacher Style)

Think of asymptotic notations like **traffic signs** for algorithm performance. When you're driving (running code), **Big-O** is your speed limit sign - "Don't go FASTER than this!" (upper bound). **Big-Omega** is "You can't go SLOWER than this minimum speed" (lower bound). **Big-Theta** says "Your speed will be EXACTLY in this range."

Real-life analogy: Planning a Mumbai party. Big-O = "At most 200 people can fit" (even if only 50 show up). Big-Omega = "At least 20 people will come" (guaranteed minimum). Big-Theta = "Exactly 100-120 people expected."

WHY used: Computers vary (slow phone vs fast server), inputs vary (sorted vs random). Notations give **guaranteed bounds** regardless of hardware/input randomness - crucial for FAANG scale (billions of users).

Step-by-step intuition:

1. Write operations as function $f(n) = 3n^2 + 2n + 1$
2. Ask: "What's the MAXIMUM growth?" $\rightarrow n^2$ dominates
3. Notations formally say $f(n) \leq \text{some } g(n)$ for large n

2. Core Concepts (Deep Understanding)

Formal Definitions (for large n , $n \geq n_0$):

Notation	Mathematical Definition	Meaning	Example
Big-O	$f(n) \leq c \cdot g(n)$	f grows $\leq g$	$3n^2 = O(n^2)$ ($c=4$ works)
Big-Ω	$f(n) \geq c \cdot g(n)$	f grows $\geq g$	$3n^2 = \Omega(n)$ ($c=1$ works)
Big-Θ	O AND Ω both true	f grows $= g$	$n^2 = \Theta(n^2)$
Little-o	$f(n)/g(n) \rightarrow 0$	f grows $\ll g$	$n = o(n^2)$
Little-ω	$f(n)/g(n) \rightarrow \infty$	f grows $\gg g$	$n^2 = \omega(n)$

Internal Logic:

- $c > 0$ constant, n_0 threshold where inequality holds
- **Drop constants/low terms** because for large n : $3n^2 + 2n \approx n^2$
- **Little versions** are stricter (no constant factor allowed)

Key Detail: Big-O is **upper bound** (safety guarantee), not "tight" or "exact."

3. Examples (Very Important)

Example 1: Binary Search

Time: $\log n$ comparisons

$\log n = O(\log n), \Theta(\log n), \Omega(\log n)$

NOT $O(1)$, NOT $O(n)$

Example 2: Linear Search

Time: n comparisons (worst case)

$n = O(n), \Omega(n), \Theta(n)$

$n \neq O(\log n), n \neq O(1)$

Example 3: Merge Sort

Time: $n \log n$

$n \log n = O(n \log n), \Omega(n \log n), \Theta(n \log n)$

$n \log n \neq O(n), n \log n \neq O(n^2)$

Example 4: Constant Time

Array access: `arr[i]`

$1 = O(1), \Theta(1), \Omega(1)$

4. Important Interview Notes ⚠️

Most Asked:

Q: "Is $n^2 = O(n^3)$?" A: YES ($n^2 \leq 1 \cdot n^3$ for $n \geq 1$)

Q: "Merge sort $O(n \log n)$, Quick sort?" A: "Average $\Theta(n \log n)$, Worst $O(n^2)$ "

Tricky Observations:

- $n = O(n^2)$ TRUE, but $n^2 \neq O(n)$ FALSE
- All functions are $O(\text{of something larger})$
- Θ implies both O and Ω

Common Confusions:

✗ "Bubble sort is $O(n)$ " $\rightarrow$ NO, worst $O(n^2)$

✗ "Binary search $O(n)$ " $\rightarrow$ NO, $O(\log n)$

☑ Always specify case: "Quick sort average $\Theta(n \log n)$ "

FAANG Tip: Draw growth graph: $O(1)$ flat, $O(\log n)$ slow rise, $O(n)$ linear, $O(n^2)$ parabola, $O(2^n)$ rocket.

5. Problem Identification

No specific problems, adding **notation-focused FAANG problems**:

1. **Prove Complexities** (Merge Sort recurrence)
2. **Compare Notations** (Which is true?)
3. **Tight Bounds** (Find Θ for given function)
6. Solve Each Problem (VERY IMPORTANT)

◇ Problem 1: Prove Merge Sort Complexity

Problem Statement: Show merge sort $T(n) = 2T(n/2) + O(n)$ solves to $\Theta(n \log n)$.

Intuition: Each level does n work, $\log n$ levels deep.

Pattern Used: Recurrence Trees / Master Theorem.

Brute Force (Unrolling)

$$\begin{aligned} T(n) &= 2T(n/2) + cn \\ &= 2[2T(n/4) + c(n/2)] + cn = 4T(n/4) + 2cn + cn \\ &= \dots = nT(1) + cn \log n = O(n \log n) \end{aligned}$$

Why tedious for general case.

Optimized (Master Theorem)

Key idea: $T(n) = aT(n/b) + f(n)$, here $a=2, b=2, f(n)=O(n)$

Case 2: $f(n) = \Theta(n^{\log_b a}) = \Theta(n^1)$

$\rightarrow T(n) = \Theta(n \log n)$

Code (Verification)

```
// Count levels in merge sort
public class MergeSortLevels {
    static int levels = 0;
    public static void mergeSort(int[] arr, int l, int r) {
        if(l >= r) return;
```

```

    int mid = (l+r)/2;
    levels++; // Each recursive level
    mergeSort(arr, l, mid);
    mergeSort(arr, mid+1, r);
    // merge O(n)
}
}

```

Detailed Dry Run ★

arr=, n=4^{[26][27][28][29]}

Level 1: split,^{[27][28][29][26]}

Level 2: split,,, → 3 levels = $\log_2 4 + 1$ ^{[28][29][26][27]}

Each level ~n work → n log n.

Edge Cases: n=1 (T=0), power of 2 vs odd n.

◇ Problem 2: Notation Comparison

Problem Statement: For $f(n)=n^2 + n \log n$, prove notations.

Intuition: Dominant term n^2 controls bounds.

Pattern Used: Limit Definition.

Code (Mathematical Proof)

```

// Pseudo-code for limits
f(n) = n2 + n log n
g1(n) = n2 → f(n)/g1(n) = 1 + (log n)/n → 1 (Θ)
g2(n) = n3 → f(n)/g2(n) → 0 (o)
g3(n) = n → f(n)/g3(n) → ∞ (ω)

```

Detailed Dry Run ★

n=1000: f=1M + 10K = 1.01M

g1=n²=1M → ratio ≈ 1 ✓ $\Theta(n^2)$

g2=n³=1B → ratio ≈ 0.001 ✓ $o(n^3)$

g3=n=1000 → ratio ≈ 1010 ✓ $\omega(n)$

◇ Problem 3: Find Tight Bound

Problem Statement: $f(n)=3n^2 + 2n + 1 \rightarrow ?$

// $\Theta(n^2)$ because:

$f(n) \leq 6n^2$ (for $n \geq 1$) $\rightarrow O(n^2)$

$f(n) \geq 3n^2 \rightarrow \Omega(n^2)$

$\rightarrow \Theta(n^2)$

7. Topic Summary (Quick Revision)

Key Points:

- **Big-O:** Upper $\leq$, **Ω :** Lower $\geq$, **Θ :** Both = tight
- **Little-o/ ω :** No constants, stricter
- **Drop constants/low terms** for large n

Patterns/Tricks:

Growth order: $O(1) < \log n < n < n \log n < n^2 < 2^n$

To prove $f=O(g)$: Show $f(n) \leq c \cdot g(n)$ for large n

FAANG: Always ask "worst/average/best?" + "prove it"

Topic 2 complete! Ready for Topic 3 (Complexity in Practical Programming)? Say "next" 🚀

*
**

next

#####

TOPIC: Complexity in Practical Programming

#####

1. Simple & Detailed Theory (Teacher Style)

Imagine you're cooking biryani in your Kolhapur kitchen. **Space Complexity** is like asking "Do I have enough pots, spices, and counter space?" – it's ALL memory your recipe uses. **Recursive programs** are like calling your mom for each step – each call stacks up (uses more phone lines/memory) until done.

Real-life analogy: Iterative loop = one notepad. Recursion = stack of notepads (one per call). When stack gets too deep → **StackOverflowError** (like too many plates falling!).

WHY used: FAANG handles billions of users. $O(n)$ space with $n=10^9 = 8\text{GB}$ memory → crash! Must optimize space ruthlessly.

Step-by-step intuition:

1. **Variables** = few bytes ($O(1)$)
2. **Arrays/trees** = $O(n)$
3. **Recursion depth** = extra $O(\text{depth})$ space
4. **Total space** = input + aux + recursion stack

2. Core Concepts (Deep Understanding)

Space Breakdown:

Total Space = Input Space + Auxiliary Space + Recursion Stack

1. **Variable Space:** Primitives (int=4B), references (8B) → $O(1)$
2. **Data Structures:** Array= $O(n)$, Tree= $O(n \text{ nodes})$
3. **Recursion Stack:** Each call = local vars + params + return addr → **$O(\text{depth})$**

Call Stack Mechanics:

fib(4):
fib(4) calls → fib(3) + fib(2)
fib(3) calls → fib(2) + fib(1) [stack: fib4, fib3, fib2, fib1 = $O(4)$]
fib(1) returns 1
... continues until all unwind

Key Details:

- **Tail recursion** optimized (some langs) $\rightarrow O(1)$ stack
- **Java default stack** = 1MB $\rightarrow \sim 10^5$ recursive calls max
- **Input space** ALWAYS counted unless "auxiliary space only"

3. Examples (Very Important)

Example 1: Iterative vs Recursive Sum

```
// Iterative: O(n) input + O(1) aux = O(n)
int sumIter(int[] arr) {
    int sum = 0;      // O(1)
    for(int x : arr) sum += x;
    return sum;
}

// Recursive: O(n) input + O(n) stack = O(n)
int sumRec(int[] arr, int i) {
    if(i == arr.length) return 0;
    return arr[i] + sumRec(arr, i+1); // stack depth = n
}
```

Example 2: Tree Traversal

```
// DFS Recursive: O(n) input + O(h) stack, h=height
void dfs(TreeNode node) {
    if(node == null) return;
    visit(node);
    dfs(node.left); // stack = O(h)
    dfs(node.right);
}
```

Example 3: Factorial

```
// Recursive: O(n) stack
long fact(int n) {
    if(n <= 1) return 1;
    return n * fact(n-1); // n stack frames
}
```

4. Important Interview Notes ⚠

Most Asked:

Q: "Recursive Fibonacci space?" A: "O(n) stack, not O(1)!"

Q: "In-place merge sort?" A: "No, needs O(n) temp array"

Q: "Why iterative better?" A: "O(1) stack vs O(n)"

Tricky:

✗ Merge sort space = $O(\log n)$ → NO, O(n) temp arrays!

☑ DFS balanced tree = $O(\log n)$, skewed = $O(n)$

Common Mistakes:

✗ Forgetting recursion stack space

✗ Counting input as "auxiliary" space

☑ Always: "Total O(n), auxiliary O(h)"

FAANG Tip: Interviewer: "Optimize space!" → Convert recursion → iteration + stack simulation.

5. Problem Identification

Standard FAANG problems testing space analysis:

1. **Reverse Linked List** (iterative vs recursive space)
2. **Maximum Depth of Binary Tree** (space optimization)

3. Merge Two Sorted Lists (in-place vs recursive)

6. Solve Each Problem (VERY IMPORTANT)

◇ Problem 1: Reverse Linked List

Problem Statement: Reverse singly linked list. Input: head of list, Output: new head. Constraints: $1 \leq n \leq 5 \times 10^4$.

Intuition: Iterative: 3 pointers. Recursive: stack simulates iteration.

Pattern Used: Pointer Manipulation.

Iterative (Optimal Space)

```
ListNode reverseList(ListNode head) {  
    ListNode prev = null, curr = head, next = null;  
    while(curr != null) {  
        next = curr.next; // save next  
        curr.next = prev; // reverse  
        prev = curr;      // move prev  
        curr = next;      // move curr  
    }  
    return prev; // O(1) space!  
}
```

Detailed Dry Run ★

1→2→3→4→null

```
curr=1: next=2, 1→null, prev=1, curr=2  
curr=2: next=3, 2→1, prev=2, curr=3  
curr=3: next=4, 3→2, prev=3, curr=4  
curr=4: next=null, 4→3, prev=4, curr=null  
return 4→3→2→1
```

Recursive (Stack Space)

```

ListNode reverseList(ListNode head) {
    if(head == null || head.next == null) return head;
    ListNode newHead = reverseList(head.next); // stack O(n)
    head.next.next = head;
    head.next = null;
    return newHead;
}

```

Space: $O(n)$ stack vs $O(1)$ iterative → **Always prefer iterative!**

◇ Problem 2: Maximum Depth of Binary Tree

Problem Statement: Find max depth (root=1). $n \leq 10^4$.

Intuition: Recursion depth = tree height.

```

int maxDepth(TreeNode root) {
    if(root == null) return 0;
    int left = maxDepth(root.left); // stack O(h)
    int right = maxDepth(root.right);
    return 1 + Math.max(left, right);
}

```

Space Analysis:

- **Balanced tree:** $O(\log n)$ stack
- **Skewed tree:** $O(n)$ stack
- **Total:** $O(h)$ where h = height

Iterative BFS (Better Space)

```

int maxDepth(TreeNode root) {
    if(root == null) return 0;
    Queue q = new LinkedList<>();
    q.offer(root);
    int depth = 0;
    while(!q.isEmpty()) {
        int size = q.size();
        for(int i=0; i<size; i++) {

```

```

        TreeNode node = q.poll();
        if(node.left != null) q.offer(node.left);
        if(node.right != null) q.offer(node.right);
    }
    depth++;
}
return depth; // O(w) space, w=width ≤ n
}

```

◆ Problem 3: Merge Two Sorted Lists

Problem Statement: Merge list1, list2 into sorted list.

Recursive Space Trap:

```

ListNode mergeTwoLists(ListNode l1, ListNode l2) {
    if(l1 == null) return l2;
    if(l2 == null) return l1;
    if(l1.val < l2.val) {
        l1.next = mergeTwoLists(l1.next, l2); // stack O(n)
        return l1;
    }
    l2.next = mergeTwoLists(l1, l2.next);
    return l2;
}

```

Iterative (O(1) space):

```

ListNode mergeTwoLists(ListNode l1, ListNode l2) {
    ListNode dummy = new ListNode(0);
    ListNode tail = dummy;

    while(l1 != null && l2 != null) {
        if(l1.val < l2.val) {
            tail.next = l1; l1 = l1.next;
        } else {
            tail.next = l2; l2 = l2.next;
        }
    }
}

```

```

    tail = tail.next;
}
tail.next = (l1 != null) ? l1 : l2;
return dummy.next; // O(1) space!
}

```

7. Topic Summary (Quick Revision)

Key Points:

Total Space = Input + Aux + Recursion Stack $O(\text{depth})$

Iterative > Recursive for space

BFS queue = $O(\text{width})$, DFS stack = $O(\text{height})$

Patterns/Tricks:

FAANG: "Optimize space!" → Convert recursion → iteration

Tree space: Always specify balanced $O(\log n)$ vs skewed $O(n)$

StackOverflow = recursion too deep → use iteration

Topic 3 complete! Say "next" for Topic 4 (Solving Recurrence Relations) 🚀

next

#####

TOPIC: Solving Recurrence Relations

#####

1. Simple & Detailed Theory (Teacher Style)

Imagine you're a factory manager in Kolhapur making samosas. **Recurrence relations** are like your production plan:

"To make 10 samosas, I need 2 workers each making 5 samosas, plus 10 minutes mixing dough."

$T(n) = 2T(n/2) + n$ means: Solve 2 subproblems of half size, plus n work to combine.

Real-life analogy: Binary search = "Check middle, then search ONE half" $\rightarrow T(n) = T(n/2) + 1$

Merge sort = "Split into 2 halves, solve both, merge" $\rightarrow T(n) = 2T(n/2) + n$

WHY used: ALL divide-and-conquer algorithms (Merge Sort, Quick Sort, Binary Search) create recurrences. FAANG asks "Prove your $O(n \log n)$ claim!" $\rightarrow$ Solve recurrence.

Step-by-step intuition:

1. **Write recurrence** from algorithm structure
2. **Unroll** or use **Master Theorem** for closed form
3. **Verify** with small inputs

2. Core Concepts (Deep Understanding)

Types of Recurrences:

1. Divide & Conquer (Master Theorem):

$$T(n) = a T(n/b) + f(n)$$

a = #subproblems, b = size reduction, $f(n)$ = combine work

Master Theorem Cases:

Case	Condition	Solution
1	$f(n) = O(n^{(\log_b a - \epsilon)})$	$T(n) = \Theta(n^{(\log_b a)})$
2	$f(n) = \Theta(n^{(\log_b a)} \log^k n)$	$T(n) = \Theta(n^{(\log_b a)} \log^{(k+1)} n)$
3	$f(n) = \Omega(n^{(\log_b a + \epsilon)})$ + regularity	$T(n) = \Theta(f(n))$

2. Linear Recurrences:

Homogeneous: $T(n) = c_1 T(n-1) + c_2 T(n-2)$

Non-homogeneous: $T(n) = c_1 T(n-1) + \dots + g(n)$

3. Akra-Bazzi (Advanced):

$$T(x) = g(x) + \sum a_i T(b_i x + h_i(x))$$

p = unique solution to $\sum a_i b_i^p = 1$

$$T(x) = \Theta(x^p (1 + \int [g(u)/u^{(p+1)}] du))$$

3. Examples (Very Important)

Example 1: Binary Search

$$T(n) = T(n/2) + 1$$

$$a=1, b=2, f(n)=1$$

$$\log_b a = 0, f(n)=n^0$$

$$\text{Case 2} \rightarrow T(n) = \Theta(\log n)$$

Example 2: Merge Sort

$$T(n) = 2T(n/2) + n$$

$$a=2, b=2, f(n)=n$$

$$\log_b a = 1, f(n)=\Theta(n^1)$$

$$\text{Case 2} \rightarrow T(n) = \Theta(n \log n)$$

Example 3: Fibonacci

$$T(n) = T(n-1) + T(n-2), T(0)=0, T(1)=1$$

$$\text{Characteristic eq: } r^2 - r - 1 = 0$$

$$r = (1 \pm \sqrt{5})/2 \rightarrow T(n) = A \phi^n + B (-\phi)^{-n}$$

$$\rightarrow T(n) \approx \phi^n / \sqrt{5} = O(\phi^n) \approx O(1.618^n)$$

4. Important Interview Notes

Most Asked:

Q: "Merge sort recurrence?" $\rightarrow T(n)=2T(n/2)+n = \Theta(n \log n)$

Q: "Binary search?" $\rightarrow T(n)=T(n/2)+1 = \Theta(\log n)$

Q: "Fibonacci?" $\rightarrow T(n)=T(n-1)+T(n-2) = O(\phi^n)$

Tricky:

Master Theorem floor/ceiling? $\rightarrow$ Assume $n/2$ works

Case 2 has $\log^k n \rightarrow k=0$ gives $\Theta(n^{\log_b a} \log n)$

Regularity for Case 3: $a f(n/b) \leq c f(n), c < 1$

Common Mistakes:

✗ $T(n)=2T(n/2)+n$ is $O(n^2) \rightarrow$ NO! $\Theta(n \log n)$

✗ Fibonacci is $O(2^n) \rightarrow$ YES, but tight is $O(\phi^n)$

✓ Always verify with $n=1,2,4,8,\dots$

5. Problem Identification

FAANG-level recurrence problems:

1. **Merge Sort Analysis** (Master Theorem)
2. **nth Fibonacci Closed Form** (Linear homogeneous)
3. **Strassen Matrix Multiplication** (Akra-Bazzi)

6. Solve Each Problem (VERY IMPORTANT)

◇ Problem 1: Merge Sort Analysis

Problem Statement: Prove $T(n)=2T(n/2)+cn = \Theta(n \log n)$

Intuition: $\log n$ levels, each cn work $\rightarrow cn \log n$

Pattern Used: Master Theorem / Recursion Tree

Brute Force (Unrolling)

$$\begin{aligned}T(n) &= 2T(n/2) + cn \\&= 2[2T(n/4) + c(n/2)] + cn = 4T(n/4) + 2cn + cn \\&= \dots = n T(1) + cn \log n = \Theta(n \log n)\end{aligned}$$

Master Theorem

$$\begin{aligned}a &= 2, b = 2, f(n) = cn = \Theta(n) \\ \log_b a &= \log_2 2 = 1 \\ \text{Case 2: } f(n) &= \Theta(n^{\log_b a}) \\ \rightarrow T(n) &= \Theta(n \log n)\end{aligned}$$

Code Verification

```
// Count operations at each level
public class MergeSortLevels {
    static int totalWork = 0;
    public static void mergeSort(int[] arr, int l, int r) {
        if(l >= r) return;
        int mid = (l + r) / 2;
        mergeSort(arr, l, mid);
        mergeSort(arr, mid + 1, r);
        totalWork += (r - l + 1); // merge work
    }
}
```

Detailed Dry Run ★ $n=8$

Level 0: 8 work
Level 1: $2 * (4) = 8$ work
Level 2: $4 * (2) = 8$ work
Level 3: $8 * (1) = 8$ work
Total: $8 * 4 = 32 = 8 \log 8 \checkmark$

◇ Problem 2: nth Fibonacci Closed Form

Problem Statement: Solve $T(n) = T(n-1) + T(n-2)$, $T(0) = 0, T(1) = 1$

Intuition: Linear homogeneous $\rightarrow$ characteristic equation

Pattern Used: Linear Recurrence Solving

Solution Steps:

Characteristic eq: $r^2 - r - 1 = 0$

$r = [1 \pm \sqrt{5}]/2 = \phi, \psi$ (golden ratio, conjugate)

$T(n) = A \phi^n + B \psi^n$

$T(0)=0: A + B = 0 \rightarrow B = -A$

$T(1)=1: A \phi - A \psi = 1 \rightarrow A(\phi - \psi) = 1$

$\phi - \psi = \sqrt{5} \rightarrow A = 1/\sqrt{5}$

$T(n) = (\phi^n - \psi^n)/\sqrt{5}$

Code Implementation

```
public class Fibonacci {  
    private static final double PHI = (1 + Math.sqrt(5)) / 2;  
    private static final double PSI = (1 - Math.sqrt(5)) / 2;  
  
    public static long fibClosedForm(int n) {  
        double phiN = Math.pow(PHI, n);  
        double psiN = Math.pow(PSI, n);  
        return Math.round((phiN - psiN) / Math.sqrt(5));  
    }  
}
```

Detailed Dry Run ★ $n=5$

$\phi^5 \approx 11.090, \psi^5 \approx -0.090$

$(11.090 + 0.090)/\sqrt{5} \approx 11.18/2.236 \approx 5 \checkmark$

Actual: 0,1,1,2,3,5 $\checkmark$

Time Complexity: $O(\log n)$ using fast exponentiation!

♦ Problem 3: Strassen Matrix Multiplication

Problem Statement: $T(n) = 7T(n/2) + O(n^2)$

Master Theorem:

$$a=7, b=2, f(n)=n^2$$

$$\log_b a = \log_2(7) \approx 2.807$$

$$\text{Case 1: } n^2 = O(n^{(2.807-\epsilon)}) \quad \checkmark$$

$$T(n) = \Theta(n^{2.807}) \approx \Theta(n^{\log_2 7})$$

7. Topic Summary (Quick Revision)

Key Formulae:

$$\text{Master Theorem: } T(n) = aT(n/b) + f(n)$$

$$\text{Case 1: } f(n) = O(n^{(\log_b a - \epsilon)}) \rightarrow \Theta(n^{\log_b a})$$

$$\text{Case 2: } f(n) = \Theta(n^{\log_b a}) \rightarrow \Theta(n^{\log_b a} \log n)$$

$$\text{Case 3: } f(n) = \Omega(n^{(\log_b a + \epsilon)}) \rightarrow \Theta(f(n))$$

$$\text{Fibonacci: } T(n) \approx \phi^n / \sqrt{5}, \phi = (1 + \sqrt{5})/2$$

FAANG Patterns:

$$\text{Binary Search: } T(n/2) + 1 = O(\log n)$$

$$\text{Merge Sort: } 2T(n/2) + n = O(n \log n)$$

Tricks:

Always draw recursion tree first!

Verify with $n=1, 2, 4, 8$

Master Theorem > unrolling for interviews

Topic 4 complete! Say "next" for Topic 5 (NP-Completeness) 🚀

next

#####

TOPIC: Advanced Concepts (NP-Completeness)

#####

1. Simple & Detailed Theory (Teacher Style)

Imagine you're organizing a massive Kolhapur wedding with 100 guests, but you need to assign tables such that **no two enemies sit together**. This is **NP-Complete** – easy to **verify** a good seating (check neighbors), but **brutally hard** to **find** the perfect arrangement among billions of possibilities.

Real-life analogy: Sudoku puzzle. Easy to check if solved correctly (verify), impossible to solve by hand for 17x17 grids.

NP = problems where solutions can be verified quickly. **NP-Complete** = hardest NP problems – solve one, solve them all!

WHY used: FAANG asks this in **system design** interviews: "Why can't Google brute-force optimal ad placement?"

Answer: NP-Complete → need heuristics/approximation.

Step-by-step intuition:

1. **P** = solvable in polynomial time ($O(n^k)$)
2. **NP** = verifiable in polynomial time
3. **NP-Complete** = NP + "reduction" from other NP-C problems
4. **P vs NP?** \$1M question – assumed $P \neq NP$

2. Core Concepts (Deep Understanding)

P vs NP Hierarchy:

$P \subset NP$

$NP\text{-Complete} \subseteq NP\text{-Hard}$

Formal Definitions:

- **P:** Solvable by deterministic Turing machine in $O(n^k)$ [e.g., sorting]
- **NP:** Solvable by nondeterministic TM in $O(n^k)$ OR verifiable in $O(n^k)$
- **NP-Complete:** NP problem + every NP problem reduces to it in poly time
- **NP-Hard:** Every NP problem reduces to it (may not be in NP)

Reduction Proof Process:

SAT $\rightarrow$ 3-SAT $\rightarrow$ Vertex Cover $\rightarrow$... $\rightarrow$ your problem

"If I can solve YOUR problem $\rightarrow$ I can solve SAT $\rightarrow$ P=NP!"

Key Examples:

NP-Complete: 3-SAT, Traveling Salesman, Knapsack, Graph Coloring

NP-Hard: Halting Problem (not even verifiable)

Cook-Levin Theorem: SAT is first NP-Complete problem.

3. Examples (Very Important)

Example 1: 3-SAT (Boolean Satisfiability)

Input: $(x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_4 \vee \neg x_5) \wedge \dots$

Q: Assign T/F to make all true?

2^n possibilities $\rightarrow$ NP-Complete.

Example 2: Traveling Salesman Problem (TSP)

Cities + distances. Find shortest tour visiting all.

Optimal solution: $\Omega(n!)$ time $\rightarrow$ NP-Complete

Example 3: Knapsack (0/1)

weights=[2,3,4], values=[3,4,5], W=5

Max value? NP-Complete (exact solution)

Verification: Given solution, check in $O(n)$ time ✓ NP.

4. Important Interview Notes ⚠

Most Asked:

Q: "P vs NP?" A: "P=solvable fast, NP=verifiable fast. NP-Complete=hardest NP."

Q: "Give NP-Complete example?" → "SAT, TSP, Knapsack"

Q: "Why matters?" → "Can't solve optimally → use greedy/heuristics"

Tricky Observations:

- ☒ Knapsack NP-Complete (exact), DP solves in $O(nW)$ pseudo-poly
- ☒ Sorting NP-Complete → NO! $O(n \log n) \in P$
- ☒ Halting Problem NP-Hard but not NP (unverifiable)

Common Confusions:

- ☒ "NP = Nondeterministic Polynomial" → Technically yes, practically "Non-deterministic P"
- ☒ "All NP problems unsolvable" → NO! Just no known poly algorithm
- ☒ Interview: Draw $P \subset NP$ diagram + 1 reduction example

FAANG Pattern: System Design: "Why approximation for route optimization?" → TSP NP-Complete.

5. Problem Identification

FAANG-level NP-Complete problems:

1. **N-Queens** (Chessboard placement)
2. **Graph Coloring** (Chromatic number)
3. **Subset Sum** (Partition problem)

6. Solve Each Problem (VERY IMPORTANT)

◇ Problem 1: N-Queens (Decision Version)

Problem Statement: Place N queens on NxN chessboard so no two attack. Return true/false if possible. $n \leq 12$.

Intuition: Backtracking tries all placements, verifies conflicts.

Pattern Used: Backtracking / NP-Complete Reduction

Brute Force (Backtracking)

```
boolean solveNQueens(char[][] board, int row) {
    if(row == n) return true; // all queens placed

    for(int col=0; col<n; col++) {
        if(isSafe(board, row, col)) {
            board[row][col] = 'Q';
            if(solveNQueens(board, row+1)) return true;
            board[row][col] = '.'; // backtrack
        }
    }
    return false;
}
```

Time: $O(n!)$ worst case $\rightarrow$ NP-Complete.

Why NP-Complete: N-Queens reduces from SAT via constraint encoding.

Detailed Dry Run ★ $n=4$

```
row0: try col0  $\rightarrow$  safe ✓
row1: col0(conflict), col1(safe), col2(no), col3(yes? diag conflict)
Backtrack  $\rightarrow$  col2 safe ✓
Continue... Solution: [1,3,0,2]
```

◇ Problem 2: Graph Coloring ($k=3$)

Problem Statement: Color graph vertices with 3 colors, no adjacent same color.

Intuition: NP-Complete via 3-SAT reduction.

```
boolean isSafe(int v, int graph[][], int color[], int c) {
    for(int i=0; i<V; i++)
        if(graph[v][i]==1 && color[i]==c) return false;
    return true;
}

boolean graphColoring(int graph[][], int m, int color[], int v) {
    if(v == V) return true;

    for(int c=0; c<m; c++) {
        if(isSafe(v, graph, color, c)) {
            color[v] = c;
            if(graphColoring(graph, m, color, v+1)) return true;
            color[v] = -1; // backtrack
        }
    }
    return false;
}
```

Reduction Insight: 3-SAT clause $(x \vee y \vee z) \rightarrow$ triangle graph forces different colors $\rightarrow$ variable assignment.

◆ Problem 3: Subset Sum

Problem Statement: Given nums[], target. Exists subset summing to target? $n \leq 40$.

```
boolean subsetSum(int[] nums, int target) {
    Boolean[][] dp = new Boolean[nums.length+1][target+1];

    for(int i=0; i<=nums.length; i++) {
        for(int j=0; j<=target; j++) {
            if(i==0 && j==0) dp[i][j] = true;
            else if(i==0) dp[i][j] = false;
            else if(j==0) dp[i][j] = true;
            else {
                dp[i][j] = dp[i-1][j] ||
            }
        }
    }
}
```

```

        (j >= nums[i-1] && dp[i-1][j-nums[i-1]]);
    }
}
}
return dp[nums.length][target];
}

```

Space: $O(n \cdot \text{target})$ pseudo-polynomial. Exact NP-Complete.

7. Topic Summary (Quick Revision)

Key Points:

P = poly solvable NP = poly verifiable

NP-Complete = NP + universal reduction target

P=NP? → \$1M unsolved, assumed NO

FAANG Patterns:

Examples: SAT, TSP, Knapsack, N-Queens, Graph Coloring

Interview: "Why heuristics?" → NP-Complete

System Design: "Approximate solutions for optimization"

Tricks:

Draw $P \subset NP$ diagram

Know 3 examples + 1 reduction intuition

"DP solves NP-Complete in pseudo-poly time"

🎉 ALL 5 TOPICS COMPLETE! 🎉

Complete Time/Space Complexity mastery achieved! Ready for FAANG coding rounds 🚀

Ask "review" for any topic recap, or share next video summary!

ULTIMATE 30-SECOND Time Complexity Shortcut ⚡

FAANG Interview Ready! Here's the **exact algorithm** interviewers expect:

1. START → Count loops/recursion
2. MULTIPLY nested levels
3. HIGHEST dominates → Drop constants/low terms
4. DONE → State $O(?)$

🕒 THE 5 GOLDEN RULES (90% cases)

RULE 1: Single loop = $O(n)$

for i=0 to n → $O(n)$

RULE 2: Nested loops = $O(n^2)$, $O(n*m)$, etc

for i=0 to n → $O(n^2)$

for j=0 to n

RULE 3: Loop halves = $O(\log n)$

while low <= high → $O(\log n)$

mid = (low+high)/2

RULE 4: Multiple same loops = ADD them

for i=0 to n → $O(n)$

for j=0 to m → $O(n+m)$

RULE 5: Operations INSIDE = MULTIPLY

for i=0 to n → $O(n \log n)$

sort() // $\log n$

💧 CODE PATTERNS → INSTANT ANSWER

Pattern	Code Example	Time
Constant	arr[7_5]	$O(1)$
Linear	for(i=0;i<n;i++)	$O(n)$
Quadratic	for(i=0;i<n;i++) for(j=0;j<n;j++)	$O(n^2)$

Log	while(low<=high)	$O(\log n)$
Linearithmic	Arrays.sort(arr)	$O(n \log n)$
Exponential	fib(n) recursion	$O(2^n)$

🕒 STEP-BY-STEP ALGORITHM (30 seconds)

GIVEN CODE:

```
for i=0 to n
  for j=0 to n/2
    arr[i] += 1
```

STEP 1: Outer loop $\rightarrow n$ times ✓

STEP 2: Inner loop $\rightarrow n/2$ times ✓

STEP 3: Inside: $O(1)$ ✓

STEP 4: Total: $n * (n/2) * 1 = O(n^2)$ ✓

⚠️ INTERVIEW TRAPS (Don't miss these!)

✗ WRONG: "Loop runs $n/2$ times $\rightarrow O(n/2)$ "

☑️ RIGHT: " $O(n)$ - drop constants"

✗ WRONG: "Two $O(n)$ loops $\rightarrow O(2n)$ "

☑️ RIGHT: " $O(n)$ "

☑️ ALWAYS: "Worst case $O(n^2)$, $O(1)$ space"

🚀 FAANG QUICK-CHECKLIST

[] Single loop? $\rightarrow O(n)$

[] Nested loops? $\rightarrow O(n^2)$

[] Binary search? $\rightarrow O(\log n)$

[] Sort? $\rightarrow O(n \log n)$

[] Recursion depth? $\rightarrow O(\text{depth})$

[] Hashmap? $\rightarrow O(n)$ time, $O(n)$ space

💡 PRO TIP: Verbalize in Interview

"Outer loop runs n times

Inner loop runs n times

Inside is $O(1)$ work

Total: $n \times n \times 1 = O(n^2)$ "

Master this → Analyze ANY code in 10 seconds! Try on LeetCode, you'll ace complexity questions 🚀 [\[30\]](#)[\[31\]](#)[\[32\]](#)

**

🔥 RECURSION & GRAPH Time Complexity SHORTCUTS ⚡

FAANG 10-Second Rules – No Math Needed!

🌀 RECURSION: 3 Golden Rules

RULE 1: Count how input SHRINKS each call

- $n \rightarrow n-1 \rightarrow O(n)$ [Linear: factorial]
- $n \rightarrow n/2 \rightarrow O(\log n)$ [Binary search]
- 2 calls $n/2 \rightarrow O(n \log n)$ [Merge sort]

RULE 2: #calls $\times$ work_per_call = TOTAL

fib(n): 2^n calls $\times O(1) = O(2^n)$ 🧠

RULE 3: SPACE = recursion DEPTH

- Tree height $h \rightarrow O(h)$ stack
- Balanced tree $\rightarrow O(\log n)$
- Skewed $\rightarrow O(n)$

📊 RECURSION PATTERNS CHEAT SHEET

Pattern	Recurrence	Time	Space	Example
Linear	$T(n-1) + O(1)$	$O(n)$	$O(n)$	Factorial

Binary Search	$T(n/2) + O(1)$	$O(\log n)$	$O(\log n)$	Search
Merge Sort	$2T(n/2) + O(n)$	$O(n \log n)$	$O(n)$	Sorting
Fibonacci	$T(n-1) + T(n-2)$	$O(2^n)$	$O(n)$	Naive fib
Tree Traversal	$2T(\text{child}) + O(1)$	$O(n)$	$O(h)$	DFS

🔥 QUICK TEST:

```
int mystery(int n) {
    if(n <= 1) return 1;
    return mystery(n-1) + mystery(n-1); // ?
}
```

Ans: 2^n calls $\rightarrow O(2^n)$ 🚀

🌐 GRAPHS: 4 Magic Formulas

$V = \text{\#nodes}$ $E = \text{\#edges}$

- 1 TRAVERSAL (BFS/DFS): $O(V + E)$
- 2 Single Source Shortest Path:
 - Unweighted: $O(V + E)$ [BFS]
 - Weighted+: $O((V+E) \log V)$ [Dijkstra]
- 3 ALL Pairs: $O(V^3)$ [Floyd]
- 4 MST: $O(E \log V)$ [Kruskal/Prim]

📊 GRAPH ALGO SHORTCUTS

Algorithm	Time Complexity	Space
BFS/DFS	$O(V + E)$	$O(V)$
Dijkstra	$O((V+E) \log V)$	$O(V)$
Bellman-Ford	$O(VE)$	$O(V)$
Floyd-Warshall	$O(V^3)$	$O(V^2)$

Kruskal	$O(E \log E)$	$O(E+V)$
Topological Sort	$O(V + E)$	$O(V)$

💡 **PRO TIP:** Sparse graph? $E \approx V$. Dense? $E \approx V^2$. Always say $O(V + E)$ for traversal!

🎮 RECURSION Dry Run Shortcut

QUICK METHOD: Count LEAVES in recursion tree

Merge Sort $n=8$:

1 call (level 0)

/ \

2 calls (level 1)

/ \ / \

4 calls (level 2) $\rightarrow$ 4 levels = $\log n$

Each level: n work $\rightarrow n \log n$ ✓

⚠️ INTERVIEW TRAPS

✗ "DFS is $O(V^2)$ " $\rightarrow$ NO! $O(V + E)$

✗ "Recursion fib is $O(n)$ " $\rightarrow$ NO! $O(2^n)$

✗ "Tree recursion is $O(n^2)$ " $\rightarrow$ NO! $O(n)$

✓ "DFS balanced tree: $O(n)$ time, $O(\log n)$ space"

🚀 30-SECOND INTERVIEW ANSWER

Q: "Tree DFS complexity?"

A: "Time $O(n)$ visits each node once.

Space $O(h)$: balanced $O(\log n)$, skewed $O(n)$ stack.

Use iterative BFS if space-tight: $O(w)$ queue width." ✓

🔗 PRACTICE PATTERNS

// PATTERN RECOGNITION

1 for node in graph: DFS(node) $\rightarrow O(V+E)$

- 2) $\text{mystery}(n/2) \rightarrow O(\log n)$
- 3) $2 * \text{mystery}(n/2) \rightarrow O(n \log n)$
- 4) $\text{mystery}(n-1) \rightarrow O(n)$

Master these → Calculate ANY recursion/graph complexity in 5 seconds! 

Try this: DFS on graph with cycles? Ans: $O(V+E)$ with visited array 

**

COMPLETE Time Complexity Cheat Sheet

Best | Average | Worst Case – FAANG Ready ⚡

Algorithm/Data Structure	Best Case	Average Case	Worst Case	Space	When Worst Happens
Array Access	$O(1)$	$O(1)$	$O(1)$	$O(1)$	Always
LinkedList Access	$O(1)$ (head)	$O(n)$	$O(n)$	$O(1)$	End of list
HashMap Get/Put	$O(1)$	$O(1)$	$O(n)$	$O(n)$	All keys hash to same bucket
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$	Always (sorted array)
Linear Search	$O(1)$	$O(n)$	$O(n)$	$O(1)$	Element at end/absent
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Reverse sorted
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	Always
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Reverse sorted
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Always
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	Already sorted + bad pivot

Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	Always
Counting Sort	$O(n + k)$	$O(n + k)$	$O(n + k)$	$O(k)$	$k = \text{range of values}$
Radix Sort	$O(nk)$	$O(nk)$	$O(nk)$	$O(n + k)$	$k = \text{\#digits}$

GRAPH Algorithms

Algorithm	Best	Average	Worst	Space	Notes
BFS/DFS	$O(V+E)$	$O(V+E)$	$O(V+E)$	$O(V)$	Single component
Dijkstra	$O((V+E)\log V)$	$O((V+E)\log V)$	$O((V+E)\log V)$	$O(V)$	Priority queue
Bellman-Ford	$O(VE)$	$O(VE)$	$O(VE)$	$O(V)$	Negative weights
Floyd-Warshall	$O(V^3)$	$O(V^3)$	$O(V^3)$	$O(V^2)$	All pairs
Kruskal MST	$O(E \log E)$	$O(E \log E)$	$O(E \log E)$	$O(E+V)$	Union-Find
Prim MST	$O((V+E)\log V)$	$O((V+E)\log V)$	$O((V+E)\log V)$	$O(V)$	Priority queue

TREE/Recursion Patterns

Pattern	Time	Space	Example
Balanced Tree Traversal	$O(n)$	$O(\log n)$	BST DFS/BFS
Skewed Tree	$O(n)$	$O(n)$	LinkedList-like
Binary Search Recursion	$O(\log n)$	$O(\log n)$	Find element
Merge Sort	$O(n \log n)$	$O(n)$	Divide+Merge
Naive Fibonacci	$O(2^n)$	$O(n)$	Tree explosion

GROWTH RATE ORDER (Memorize!)

$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n) < O(n!)$

Constant Logarithmic Linear Linearithmic Quadratic Exponential Factorial

⚠ INTERVIEW GOLDEN RULES

- 1 ****Always say WORST CASE first****: " $O(n^2)$ time, $O(1)$ space"
- 2 ****QuickSort****: "Avg $O(n \log n)$, worst $O(n^2)$ - already sorted"
- 3 ****HashMap****: "Avg $O(1)$, worst $O(n)$ - hash collisions"
- 4 ****Graphs****: ****ALWAYS $O(V+E)$ **** for traversal, never $O(V^2)$!
- 5 ****Space****: Include recursion stack $O(\text{depth})$

🔗 FAANG QUICK IDENTIFY

- ☒ $O(1)$ → Array access, HashMap avg
- ☒ $O(\log n)$ → Binary search, BST ops
- ☒ $O(n)$ → Single loop, LinkedList, Tree traversal
- ☒ $O(n \log n)$ → Sorting (merge/quick/heap), Heaps
- ☒ $O(n^2)$ → Nested loops, Bubble/Insertion
- ☒ $O(V+E)$ → BFS/DFS Graphs

💡 PRO TIP: Verbal Pattern

"Visits each [node/edge] once → $O(n)$ "

Each level n work, $\log n$ levels → $O(n \log n)$

n choices, n positions → $O(n^2)$ "

Print this table → Instant FAANG complexity mastery! 🚀 [\[49\]\[50\]\[51\]\[52\]](#)

**

1. <https://www.scribd.com/document/941750154/DSA-Interview-Prep-6Month-Roadmap>
2. https://www.reddit.com/r/leetcode/comments/1ilys2e/how_to_do_dsa_to_crack_faang_companies_especiall_y/
3. <https://www.designgurus.io/answers/detail/which-is-the-best-youtube-channel-to-learn-dsa>
4. <https://www.youtube.com/watch?v=xP3x4K4Rvi4>

5. https://www.linkedin.com/posts/srishtik-dutta-890587160_faang-interview-preparation-complete-dsa-activity-7328260299973636096-LOe4
6. <https://www.youtube.com/watch?v=bfEHKYP0lyU>
7. <https://www.youtube.com/watch?v=hwpNzen0z0w>
8. <https://github.com/AkashSingh3031/The-Complete-FAANG-Preparation>
9. <https://www.youtube.com/watch?v=9zj5sZpTjk8>
10. <https://www.youtube.com/watch?v=64L0xcPbguA>
11. <https://www.linkedin.com/pulse/what-do-we-mean-worst-case-performance-algorithm-faysal-ahmed-wws0e>
12. <https://www.geeksforgeeks.org/dsa/worst-average-and-best-case-analysis-of-algorithms/>
13. <https://www.youtube.com/watch?v=mV3wrLBbuuE>
14. <https://www.designgurus.io/answers/detail/which-is-the-best-youtube-channel-to-learn-dsa>
15. <https://github.com/AkashSingh3031/The-Complete-FAANG-Preparation>
16. <https://www.scribd.com/document/941750154/Dsa-Interview-Prep-6Month-Roadmap>
17. <https://www.youtube.com/watch?v=xP3x4K4Rvi4>
18. https://www.linkedin.com/posts/srishtik-dutta-890587160_faang-interview-preparation-complete-dsa-activity-7328260299973636096-LOe4
19. <https://www.simplilearn.com/tutorials/data-structure-tutorial/time-and-space-complexity>
20. <https://en.wikipedia.org/wiki/NP-completeness>
21. <https://www.youtube.com/watch?v=fRbHDwFvmxA>
22. <https://algotcademy.com/blog/why-you-should-practice-solving-np-complete-problems/>
23. <https://www.youtube.com/watch?v=GdC3hJDbmEA>
24. <https://algot.dev/en/best-case-and-worst-case/>
25. <https://www.jointaro.com/interview-insights/google/explain-p-np-np-hard-and-np-complete-problems-with-examples/>

26. <https://www.scribd.com/document/941750154/DSA-Interview-Prep-6Month-Roadmap>
27. <https://www.youtube.com/watch?v=xP3x4K4Rvi4>
28. <https://www.designgurus.io/answers/detail/which-is-the-best-youtube-channel-to-learn-dsa>
29. https://www.linkedin.com/posts/srishtik-dutta-890587160_faang-interview-preparation-complete-dsa-activity-7328260299973636096-LOe4
30. <https://www.geeksforgeeks.org/dsa/understanding-time-complexity-simple-examples/>
31. <https://adrianmejia.com/how-to-find-time-complexity-of-an-algorithm-code-big-o-notation/>
32. [https://www.reddit.com/r/learnprogramming/comments/v6obt3/how to figure out the time complexity of my code/](https://www.reddit.com/r/learnprogramming/comments/v6obt3/how_to_figure_out_the_time_complexity_of_my_code/)
33. <https://www.youtube.com/watch?v=XIT8yL0bB0I>
34. <https://www.youtube.com/watch?v=iGrU1UgYQkI>
35. <https://www.youtube.com/watch?v=KXAbAa1mieU>
36. https://www.w3schools.com/dsa/dsa_timecomplexity_theory.php
37. <https://www.crio.do/blog/time-complexity-explained/>
38. <https://www.youtube.com/watch?v=STL8ESuETmM>
39. <https://www.geeksforgeeks.org/dsa/how-to-analyse-complexity-of-recurrence-relation/>
40. <https://www.wscubetech.com/resources/dsa/graph-algorithms>
41. <http://www.arxiv.org/pdf/2501.09144.pdf>
42. <https://www.enjoyalgorithms.com/blog/time-complexity-analysis-of-recursion-in-programming/>
43. <https://thesciencebrigade.com/jst/article/view/452>
44. <https://read.learnyard.com/dsa/recursion-time-and-space-complexity-analysis-1/>
45. <https://memgraph.com/blog/graph-search-algorithms-developers-guide>
46. https://launchschool.com/books/advanced_dsa/read/time_and_space_complexity_recursive
47. https://www.tutorialspoint.com/graph_theory/graph_theory_time_complexity.htm

48. <https://stackoverflow.com/questions/13467674/determining-complexity-for-recursive-functions-big-o-notation>
49. <https://stackoverflow.com/questions/22438149/time-complexity-best-worst-and-average-case>
50. <https://www.geeksforgeeks.org/dsa/worst-average-and-best-case-analysis-of-algorithms/>
51. [https://en.wikipedia.org/wiki/Best, worst and average case](https://en.wikipedia.org/wiki/Best,_worst_and_average_case)
52. <https://algomaster.io/learn/dsa/best-worst-average-complexity>
53. https://www.youtube.com/watch?v=Uv2KFr2_5xY
54. https://www.w3schools.com/dsa/dsa_timecomplexity_theory.php
55. <https://www.linkedin.com/pulse/algorithm-complexity-understanding-time-space-devender-singh>
56. <https://www.bigocheatsheet.com>
57. <https://www.freecodecamp.org/news/big-o-cheat-sheet-time-complexity-chart/>
58. <https://unstop.com/blog/time-complexity-of-algorithms>